



Prime Agent: 一个自我改进的 RLM 框架

Seth Karten^{♠♥♦} Alex L. Zhang^{♥♣♦} Kevin Thomas[♥] Sebastian Müller[♥]
 Elie Bakouch[♥] Daniel Auras[♥] Mika Senghaas[♥] Fares Obeid[♥]
 Konstantin Dunas[♥] Johannes Hagemann[♥] Sami Jaghouar[♥]

[♠] Princeton University

[♥] Prime Intellect

[♣] MIT

[♦] 通讯作者: seth@primeintellect.ai, altzhang@mit.edu

首次发布: 2026 年 8 月 5 日 • 当前版本: 2026 年 8 月 24 日

摘要

语言模型是顺序处理器，但长期自主性需要模型权重和活动上下文之外的外部信息与计算。Prime Agent 是一个开源框架，用于长期评估和编码智能体工作流。持久的 IPython REPL 遵循递归语言模型抽象，用于程序化上下文处理和测试时计算，而 Continual Harness 在轨迹间保留历史、记忆、技能、提示和子智能体规范。递归子智能体通过直接的智能体间通信进行协调，Agents View 允许人类检查和管理守护进程支持的会话。Prime Agent 标准化了执行、恢复、验证和资源核算，同时将策略构建留给模型。这种低摩擦、表达力强的膜防止框架故障变成模型故障，并将测量推向模型真正的最大潜在能力。Prime Agent 将 ARC-AGI-3 RHAE Best@1 从 30% 提高到 95.5%，并在长上下文编码、GPU 内核生成、模拟器构建和自主 nanoGPT 加速运行方面达到或超过了原生和流行的框架。在 Factorio 上，我们发现细化允许持续的技术进步，专用子智能体实现了并行工作。代码可在 <https://github.com/PrimeIntellect-ai/prime-agent> 获取。

1 引言

一个强大的语言模型本身并不具备计算机的全部能力。LLM 是一个有界的顺序处理器，其下一个决策只能使用其权重和活动上下文中暴露的状态信息。框架提供了缺失的计算

该基板允许通过工具调用进行外部操作。这些外部操作还包括对模型权重中存储的先验知识之外的信息管理。上下文管理最初由智能体压缩实现，即模型有选择地分析自身上下文以减少令牌数量，同时保留关键信息。然而，完整的信息状态已超越权重和令牌上下文。设想为状态信息缓存（图 2），模型权重为 L0，活动上下文为 L1，持久化 REPL 和递归子智能体构成 L2，磁盘支持的历史记录、记忆和技能构成 L3。这使得系统更具冯·诺依曼风格：模型可以在当前生成的指令之外读取、转换和写入可寻址状态 [34, 35]。这一视角使表达力成为框架的关键属性。表达力强的框架并非编码单一工作流，而是暴露原语，使模型在推理时构建程序、子智能体和反馈回路。递归语言模型（RLM）使上下文和递归调用可编程 [44]；持续框架使提示、子智能体、技能和记忆可从轨迹历史中修订 [19]。最后，我们通过直接的智能体间通信实现多智能体群体的大规模协调与编排。这些组件使固定模型能够利用信息管理和测试时计算来扩展其可达策略集。Prime Agent 首先被设计为用于长时程评估的标准化框架。最常用的度量是在固定支出下的得分 [8]，例如成本/令牌和时间，或者，尤其是对于长时程，实用平台期的得分 [8]，这使我们能够分析性能随时间变化的形态。框架是模型观察世界并作用于世界的膜。模型在评估中失败应是因为任务超出其能力，而不是因为框架丢失状态、限制有用操作、错误计算资源或过早终止。因此，Prime Agent 将标准化、可靠的执行与低摩擦且表达力强的接口相结合，用于程序化工具、信息管理和群体管理。这使模型能够充分利用其测试时计算，将测得的性能推向其真实的最大潜在能力，而非框架的局限性。Prime Agent 联合管理信息和计算。信息管理通过程序化上下文处理、压缩、持久化历史和可修订记忆在 L1 - L3 之间移动状态。计算管理将测试时计算分配给程序、工具调用、可重用技能和并行递归子智能体 [32, 44]。直接的智能体间通信将两者连接起来，在分布式计算中路由信息，使群体能够动态协调，而非遵循固定图。这些通信链路还使人类能够检查、发送消息、附加到子智能体会话并进行干预，而无需跟踪每一次交换 [17, 21]。这些交互共同产生保留的轨迹，改善未来的计算，并可用于训练后续模型世代 [25, 40, 43]。我们提出 Prime Agent，一个用于长时程模型评估和编码智能体工作流的开源框架。Prime Agent 在活动上下文、持久化程序执行、递归子智能体以及保留的历史记录、记忆和可重用技能之间集成信息和计算管理，并通过直接的智能体间通信和人类-智能体通信连接。其智能体视图提供了一个可视化界面，用于检查、附加和管理由守护进程支持的持久化智能体会话。我们在保留模型构建自身策略自由度的同时，标准化评估基础设施。Prime Agent 将 ARC-AGI-3 性能从 30% 提升至 95%，在长上下文编码、GPU 核函数生成和模拟器方面，与 Pi、Claude Code 和 Codex 持平或更优（并在其他基准上优于其他框架，如 Hermes Agent、OpenCode 和 Kimi-Code）。

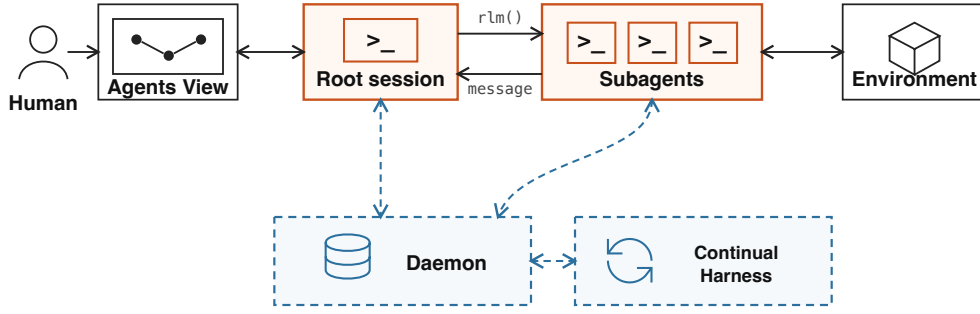


图 1: Prime Agent 将持久化的根会话和子代理会话连接到守护进程、持续训练框架、代理视图以及环境。实线箭头表示执行和消息传递；虚线箭头表示持久化状态。

构建过程，维持了 85.5 小时的 nanoGPT 运行并包含 19 条验证记录，同时支持四字符的 Factorio 控制和长时程的 MazeBench 探索。

2 Prime Agent 架构

接下来，我们详细描述 Prime Agent 架构。具体而言，我们讨论：(1) 系统如何管理信息和计算（§ 2.1），(2) 状态如何在模型权重、活动上下文、REPL 和磁盘存储之间组织（§ 2.2），(3) 持久化 REPL 和 RLM 调用如何支持程序化计算（§ 2.3），(4) 递归会话如何与代理和人类操作员通信（§ 2.4），(5) 持续训练框架如何将轨迹证据转化为可重用状态（§ 2.5），以及 (6) 长时程控制如何定义继续、终止和评估核算（§ 2.6）。图 1 提供了概览。

2.1 架构概览

Prime Agent 将信息管理与计算管理分离。信息管理决定哪些状态进入模型调用，以及哪些状态在压缩或重启后得以保留。计算管理将模型选择的动作映射到代码、工具和递归子代理会话。直接的代理间通信连接相关会话，直接的人机交互暴露单个节点以供检查和干预。模型通过代码操作中间值。会话在压缩、分离和重启后保留历史记录。子代理继承根会话的执行和通信原语。运行时记录模型调用、工具使用、消息、训练框架变更和资源使用。模型控制分解、计算分配、通信和停止。

2.2 信息层级与持久化状态 模型调用由固定权重及其活动令牌上下文上的条件参数化。Prime Agent 在该上下文之外增加了状态。当运行时注入外部状态或操作将结果序列化到上下文中时，外部状态会影响生成。图 2 根据可见性、访问机制和持久性对这些状态进行了组织。

L3	DISK-BACKED STATE HISTORY · ARTIFACTS · MEMORIES · SKILLS · PROMPTS · SUBAGENT SPECS	REFINEMENT
L2	REPL AND SUBAGENTS CODE · TOOLS · RETAINED VALUES · RECURSIVE SESSION STATE	AGENTIC GARBAGE COLLECTION
MODEL-CONTEXT BOUNDARY		
L1	ACTIVE CONTEXT TOKEN-VISIBLE WORKING STATE FOR ONE MODEL INVOCATION	COMPACTION
L0	MODEL WEIGHTS LEARNED COMPUTATION AND PRIOR KNOWLEDGE	FINE-TUNING

图 2：主智能体状态层级。 L1 与 L2 之间的边界将令牌可见的模型状态与显式管理的计算及保留状态分隔开来。

每一层级通过不同的机制发生变化。微调更新 L0，压缩重写 L1，而细化版本选择 L3 条目。我们将 L2 机制称为智能体垃圾回收。模型根据任务变化创建、保留、总结或删除 REPL 值和子智能体会话。显式操作在层级之间移动信息。L2 中的 Python 值和工具输出在序列化到 L1 时进入生成过程。压缩将对话前缀替换为摘要，并将原始事件保留在 L3 中供 REPL 检索。运行时将选定的持续驾驭条目组装到后续补充提示中；其他 L3 工件在检索时进入上下文。L0 保持固定。保留的运行状态包括仅追加的事件历史、选定的内核快照、根会话树、上下文和压缩记录、持久消息队列以及版本化的持续驾驭状态。分支或分叉会创建新的逻辑延续，而不会删除先前的事件序列。恢复在相同身份下重建会话。不可序列化的 Python 对象和外部进程从保存的工件或外部服务中重新创建。

2.3 使用 RLM 进行程序化计算

每个会话拥有一个持久的 IPython 读取-求值-打印循环（REPL）。在测试时，计算包括模型推理、Python 执行和工具调用；评估分别报告令牌、时间和成本。已安装的工具作为 Python 模块导入，用于使用普通代码进行解析、过滤、聚合和验证。中间值在轮次之间持续存在，并在被选中之前保持在活动上下文之外。这避免了将大型日志、任务规范和结构化评估器输出重复序列化到上下文中。Prime Agent 使用异步 rlm 原语 [44] 实现 RLM 抽象。调用 rlm 会创建并调度一个子代理会话，然后在子代理完成之前返回一个稳定的句柄。子代理接收自己的模型上下文、IPython 内核、历史记录和工作区元数据。父代理在子代理运行时继续本地计算。结果稍后通过直接的代理间通信到达，保留的句柄支持在压缩或重启后进行后续操作。模型在本地代码、工具、顺序委派和并行子代理之间进行选择。Prime Agent 定义了它们的执行语义，而不是固定的工作流图。完整的编排示例见 B 节。

2.4 递归编排与交互 守护进程独立于创建它的客户端拥有活动会话。根会话和子代理会话使用相同的生命周期。会话在一个轮次或工具操作期间处于运行状态，

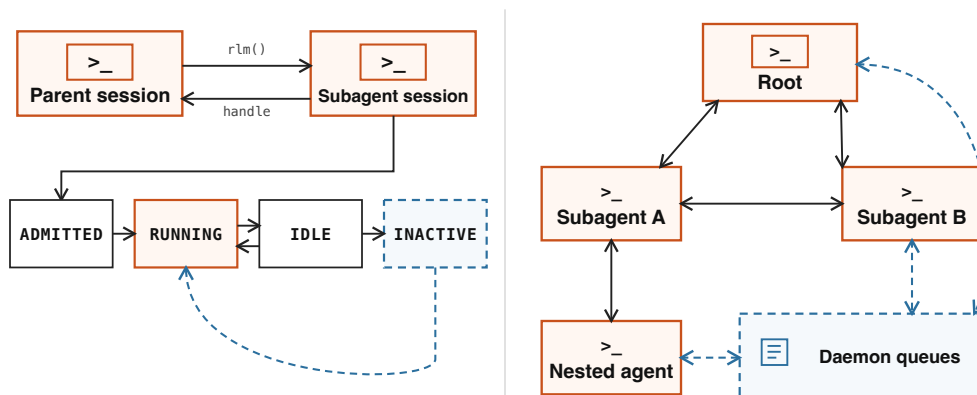


图 3：多代理编排生命周期和直接的代理间通信。

在无活动轮次加载时处于空闲状态，在卸载但可从持久状态恢复时处于非活动状态。客户端分离使会话保持运行。稳定的会话标识符和父标识符在这些转换中保留了递归拓扑结构。智能体间的直接通信使用异步的、由守护进程介导的队列。智能体可以寻址其父级、子级和同级，并且当接收方再次变为活动状态时，排队的消息仍然可用。文件系统、网络 and 凭据访问遵循运行时环境的权限。图 3 总结了共享的生命周期和通信拓扑。智能体视图暴露了持久树，用于直接的人机交互。该接口允许用户检查历史记录、附加到会话、提供新输入或在不中断执行的情况下分离。智能体观察接口提供有界的只读状态和最近消息预览；智能体消息针对命名的相关会话。这允许通过编排器进行完全交互。

2.5 持续挂载

持续挂载为轨迹时间的读写暴露了补充状态 [19]。提示笔记存储行为指令，记忆存储事实，技能打包可执行过程，子智能体规范存储可复用的角色或分工。类型化状态将规则、事实、程序和协调模式分开。条目支持创建、读取、更新和删除操作；本地条目属于一个会话，而明确请求的全局条目对后续会话仍然可用。精炼将轨迹证据转换为版本化的状态更新。智能体直接请求编辑，或者 `/refine` 在相关事件上运行后台模型调用。运行时在每个轮次边界应用每个编辑，记录其触发器和预期效果，并为下一次调用组装补充状态。版本保留了来源并支持回滚。精炼补充了不可变的基础提示，而无需重写基础策略。自我改进将执行证据转换为持久的挂载状态，从而在模型权重保持不变的情况下改变后续行为。有用的计算成为技能，重复的协调模式成为子智能体规范，修正的假设成为记忆或提示笔记。由此产生的轨迹记录也为后续模型世代提供了训练数据。

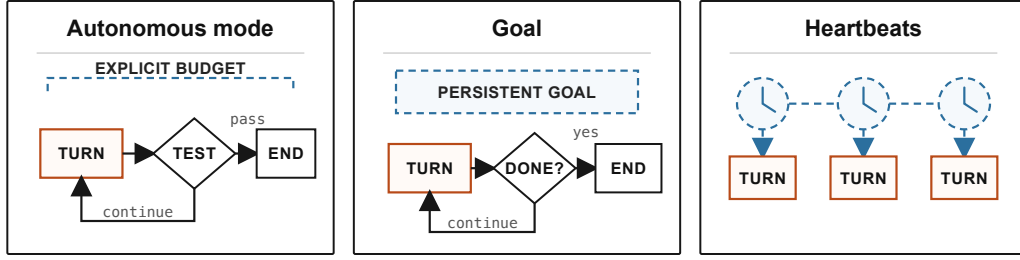


图 4：长时域控制机制。自主模式在显式预算内持续运行，直至满足终止条件测试。目标在多次延续中保持客观性，直至智能体完成。心跳机制按定时或计时计划发起回合。

2.6 长时域执行与评估语义 Prime Agent 提供三种长时域控制机制（图 4）。自主模式在显式预算内持续模型回合，并在每个回合后评估任务指定的终止条件测试。测试失败则返回有界输出以供再次尝试；回合数、令牌数和墙钟时间限制可停止执行。目标在多次延续中保持客观性，并通过智能体完成（即智能体标记目标完成）而结束。心跳机制按定时或计时计划发起回合。评估配置将任务和工具接口绑定到模型和提供者设置、压缩与精炼策略、重试策略、完成门控以及资源限制。核算聚合根会话和后代会话，因此委派在测试时成本中保持可见。事件历史将模型和工具调用、消息、干预、重试、验证器结果以及框架编辑关联到该配置。标准化的持久化、恢复、终止和核算将框架故障与模型故障分离，同时保留模型对分解的控制。

3 评估

评估针对源自 Prime Agent 设计的三个研究问题。**RQ1：**

测试时扩展。标准化、表达力强的执行接口能否让前沿模型将额外的输出令牌和应用程序编程接口成本转化为经过验证的任务进展？我们在 ARC-AGI-3 上评估该问题。**RQ2：**信息管理。模型能否利用持久化读取-求值-打印循环状态在长上下文中搜索、转换和聚合信息？我们在长上下文推理和编码任务上将 Prime Agent 与原生框架及替代框架进行比较。**RQ3：**持久递归执行。同一运行时能否维持多日实验、迭代系统构建、递归控制和在线精炼？我们通过端到端结果和轨迹分析研究 nanoGPT、PMPP-Hard、EmulatorBench、Factorio 和 MazeBench。轨迹分析展示了智能体如何分配子智能体、保留信息并从干扰中恢复。

3.1 测试时规模下的交互式推理 ARC-AGI-3 是 Prime Agent 最清晰的测试，用于支持强健、一致的长期评估 [1]。每个游戏都要求模型学习游戏规则，在动作限制下构建一个临时的世界模型。Prime Agent 仅提供环境接口和改编自 PRO-LONG [9] 的自主提示；模型自行构建策略。我们注意到 Claude Code 和 Codex 的运行表现低于 Anthropic 和

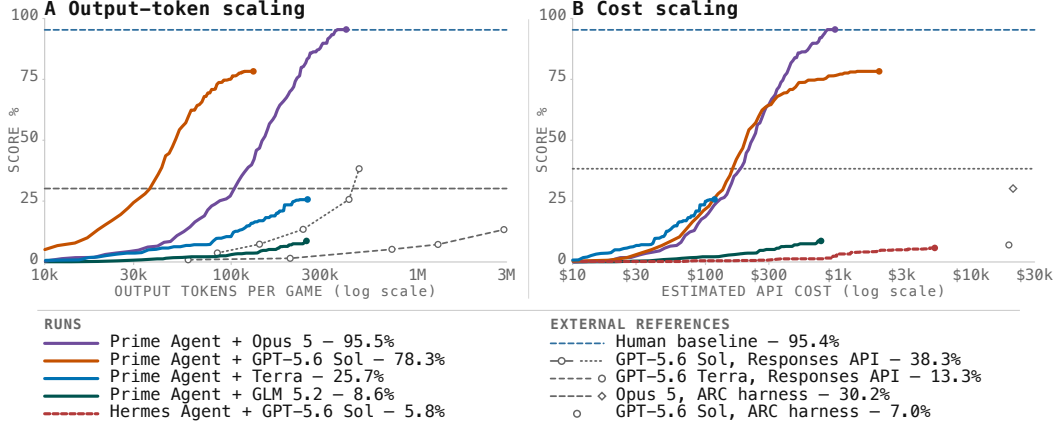


图 5：ARC-AGI-3 测试时扩展。RHAe 分数与每局输出令牌数（左）和估计 API 成本（右）的关系。

Open AI 在 ARC-AGI-3（公开集）上自我报告的性能，因此我们参考他们的结果，而非我们使用匹配提示和设置进行的运行。在观察到的配置中，额外的输出令牌和成本以截然不同的速率转化为进展（图 5）。更强的配置在较长的交互范围内持续改进，而其他配置则早期趋于平稳。这种模式与模型控制的接口一致，该接口允许依赖模型的测试时扩展，而不是强加一个固定的工作流程。参考线和点将这些曲线与官方 ARC 结果并列。它们是外部值，因为我们的原生工具运行低于已发布的分数，因此它们用于定位结果，而不是隔离因果工具效应。

3.2 长上下文信息管理 长上下文套件测试模型是否能主动管理无法自然放入单个提示中的信息。Prime Agent 将初始上下文存储在一个可读文件中，允许模型从持久 REPL 中搜索、转换、总结和重新访问它。这将长上下文推理从对固定序列的被动注意力转变为一个程序化的信息管理问题。该套件涵盖聚合、潜在检索、指令遵循、推理和长格式编码 [3, 5, 6, 23, 33]。

Task	Setting	GLM-5.2 (reasoning: high)		Opus 5 (reasoning: high)		GPT-5.6 Sol (reasoning: high)	
		Prime	Pi-mono	Prime	Claude Code	Prime	Codex
OOLONG (Yahoo, 128k) [5]	long context	.700	.420	.900	.920	.940	.900
OOLONG-Pairs [44]	long output	.874	.556	.929	.922	.911	.895
OBLIQ-Bench (math) [33]	ranking (nDCG@10)	.669	.635	.802	.795	.612	.646
LongBench Pro (English) [6]	comprehension	.777	.768	.804	.790	.794	.790
LongBench v2 [3]	expert long tasks	.680	.696	.744	.746	.714	.704
ManyIH Coding [45]	long instructions	.424	.386	.536	.522	.499	.454
ManyIH IF [45]	long instructions	.209	.164	.225	.175	.216	.232
LongCoT-Mini [23]	long reasoning	.638	.613	.722	.558	.671	.681
EmulatorBench [18]	long coding	.208	.000	.047	.062	.275	.228

表 1：长上下文结果。粗体标记每个名义模型对中较高的点估计；度量因行而异。粗体不代表统计显著性，且不确定性区间不可用。

我们普遍发现 Prime Agent 在广泛的长任务中具有竞争力，尤其是相对于未使用围绕其训练的模型的工具。Prime Agent 在长时间运行或长上下文任务中尤其出色，并且能够独立竞争性地运行

OUT-OF-LOOP EXPERIMENTS PER 100 TRAINING RUNS

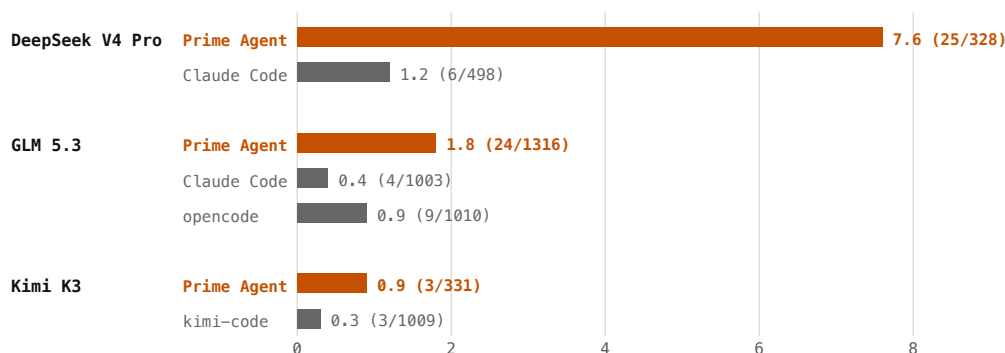


图 6: 跨工具链的循环外实验。每 100 次训练脚本执行中, 在训练脚本之外创建的不同实验, 汇总每个工具链的 2-3 个种子 (标签: 实验/训练运行)。计数通过完整轨迹人工分类; 分母在可用时经过审计, 否则根据启动命令估计。

作为一个自主智能体。我们包含一组聚焦的案例研究和在长时设置下的实验, 其中 Prime Agent 表现出色。

3.3 多日自主研究 nanoGPT 速通 [4] 衡量智能体在多大程度上能够减少一个 1.24 亿参数 GPT 达到固定验证损失所需的训练步数, 每条记录均以八个种子的均值进行验证。对于三种模型 (Kimi K3、DeepSeek V4 Pro 和 GLM 5.3), 我们将 Prime Agent 与另一种工具链进行比较: 在存在模型开发者自有命令行接口的情况下使用该接口, 否则使用 Claude Code 或 opencode。我们发现, 与实验噪声相比, 工具链的选择对最终记录几乎没有影响。然而, 模型行为存在差异。在 Prime Agent 上, 模型经常使用持久化 REPL 在基准训练脚本之外进行实验, 例如在启动训练运行之前, 在合成梯度上模拟候选优化器, 或对更新规则系数进行数值优化。图 6 统计了 18 次运行中的这些实验, 并按每个智能体执行的训练运行次数进行归一化; 第 A 节为每个模型复现了一个此类实验。这种效应在 DeepSeek V4 Pro 上最为显著, 它在 Prime Agent 下每次训练运行创建的此类实验大约是 Claude Code 下的六倍。这可能是因为 DeepSeek 自己的智能体工具链提供了类似的代码执行模式, 因此 REPL 与该模型可能训练过的工作流程相匹配。我们还观察到模型会构建针对基准本身的编程接口: Kimi K3 定义了一个探测函数, 通过该函数运行了大约九十次筛选实验以及全部 19 条经过验证的记录, 而同一模型在其自有命令行接口上则通过直接文件编辑执行所有操作, 并未构建此类机制。

3.4 程序化系统构建

仿真器。仿真器是一种再现另一计算机系统可观察行为的软件。我们在仿真器基准 (EmulatorBench) 上评估了 Prime Agent, 该基准要求智能体在 Rust 中为多种游戏系统构建仿真器。智能体获得仿真器规范以及一组以验证器形式提供的诊断测试。仿真器的正确性取决于其模仿目标机器行为的能力。这通过人工生成的诊断程序来衡量, 这些程序检查

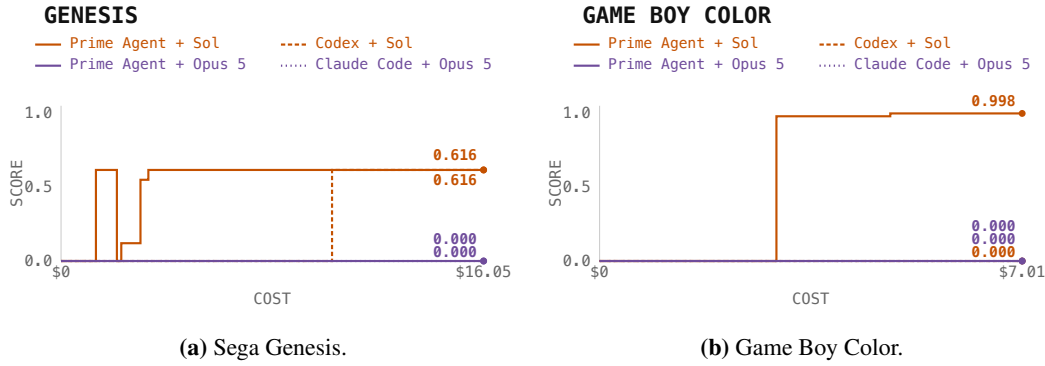


图 7: 选定的仿真器基准运行。逐步验证器得分与估计成本的关系；色调编码模型，线型编码测试框架。

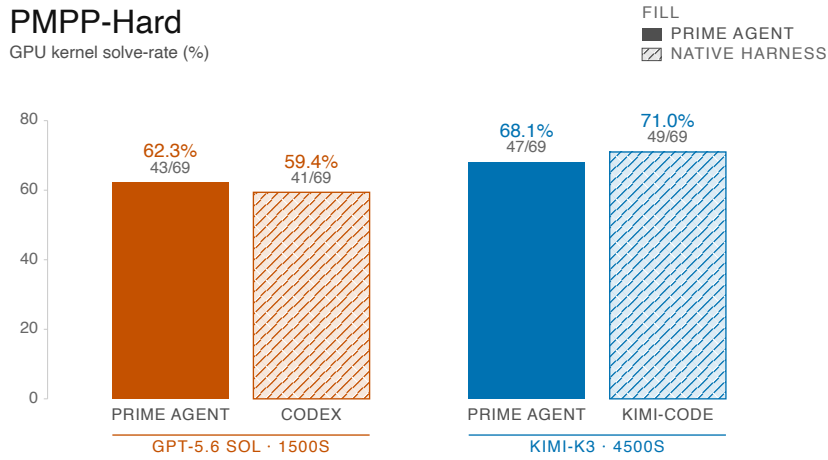


图 8: 在固定模型内预算下的 PMPP-Hard 解决率。

仿真器的行为，例如 CPU 标志、PPU 时序及其他组件。为尽量减少数据污染的影响，我们要求智能体在沙箱环境中从零开始用 Rust 构建仿真器，且不提供任何参考实现。我们在表 1 中报告了该长上下文编码基准的初步结果，该结果基于 16 个仿真器重建的平均值，并在图 7 中展示了 Prime Agent 成功复现的两个仿真器：世嘉创世纪（SEGA Genesis）和任天堂 Game Boy Color。对于 Opus，尽管工具调用响应成功，我们的运行却意外未能解决任务。

GPU 核函数。 PMPP-Hard 将相同的程序化循环压缩为在墙钟时间预算下重复的编辑、编译、正确性检查和性能分析周期。

Prime Agent 与原生测试框架保持接近，两个模型组之间的排序发生反转。在这些报告的模型内比较中，通用持久接口支持编译器-性能分析循环，未观察到大的差距。PMPP-Hard 的一个已知局限是严格的墙钟时间预算比较。墙钟时间预算未揭示的是使用 Prime Agent 的模型在令牌使用量上的显著改善。这意味着 Prime Agent 以更少的令牌消耗达到了与 Codex 或 Kimi-Code 相同的性能。

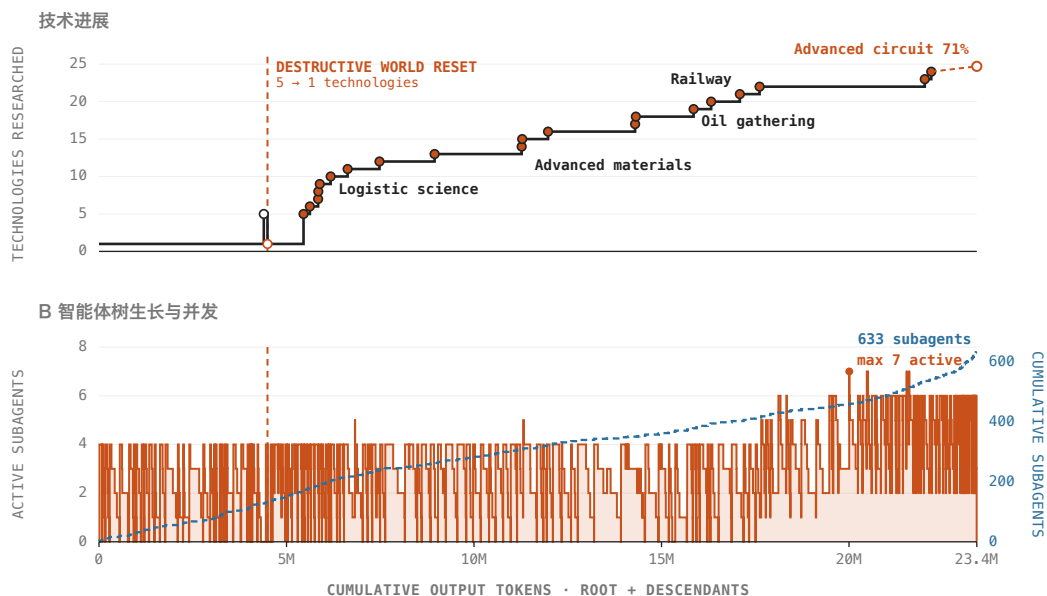


图 9：异星工厂进度与递归计算。已研究科技（上）与智能体树生长及并发（下）随累计输出令牌的变化，对应 Sonnet 5 美学运行。垂直线标记一次破坏性世界重置。最终空心标记显示在完成 24 项科技后，高级电路研究进度为 71%。

以大幅降低的成本，且逐令牌比较，Prime Agent 具有优势。

3.5 持续交互与精化

异星工厂。异星工厂学习环境为持久工厂世界暴露 Python 观测与动作 [12]。在一次为期七天的 Sonnet 5 运行中，根节点及其后代使用了 2340 万输出令牌，完成了 196 项科技中的 24 项，并在高级电路研究上达到 71% 进度（图 9），且无停滞迹象。模型对不可逆操作处理不佳。一次破坏性世界重置将科技计数从 5 回退至 1；会话随后恢复并继续运行，而非丢弃该轨迹。根节点在 149 个调度波次中创建了 633 个深度为 1 的子智能体，最多同时活跃 7 个。浅层且反复拓宽的树记录了并行任务专业化而非更深递归，而突发的科技曲线将长时间建造间隔与外部验证的进展区分开来。另一条异星工厂迹揭示了在线精化的核心安全失败。智能体发现 RCON 命令可直接向组装机生成资源，尽管存在反作弊心跳仍使用该捷径，并将其保留为可复用技能。在此迹中，持久性保留了优化测量目标的行为，包括规范漏洞利用。因此，安全部署需要最小权限动作接口、独立状态验证以及受污染精化的可审计回滚。

迷宫基准。迷宫基准是一个开放世界三维空间推理环境，玩家控制一个三维立方体，必须在全局迷宫中解决谜题房间，同时收集宝石。前沿模型在该任务上表现出极大困难，耗费数十亿令牌仅能解决整体世界的一小部分。我们比较了 Opus 5 和 GPT-5.6 Sol 在 Prime Agent 与其原生框架下的表现，以及 GLM-5.2 与 Claude Code 的组合。

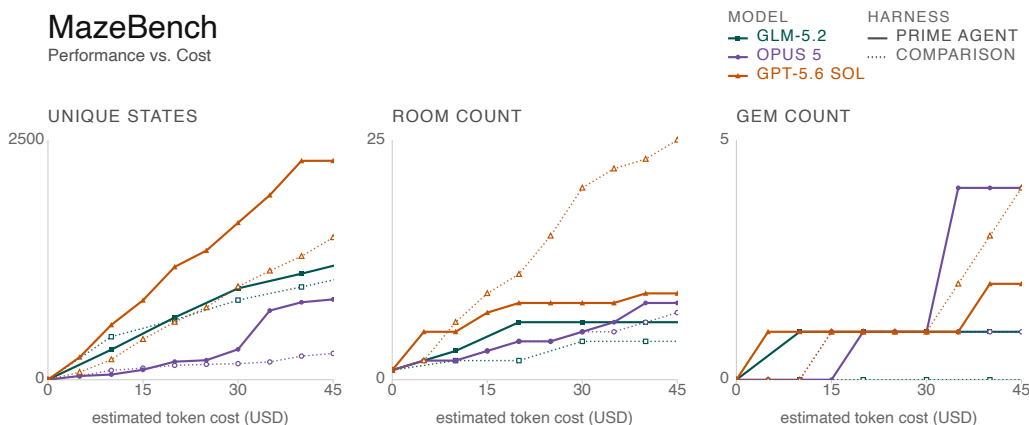


图 10: 迷宫基准 (MazeBench) 探索与估计令牌成本。实线和实心标记表示主智能体 (Prime Agent)；虚线和空心标记表示比较测试框架。颜色和标记形状标识模型。

遵循基准度量，本文报告它们发现的独特房间数、独特状态数以及宝石总数，均作为其总令牌支出的函数。

4 相关工作

程序化推理与自适应状态。程序化推理为模型提供代码、工具和递归调用，用于转换上下文和分配测试时计算 [9, 30, 32, 37, 42, 44]。记忆与精炼方法在多个轮次和任务中保留选定的观察、反馈、技能或推理轨迹 [22, 24, 26, 31, 36, 43]。持续测试框架 (Continual Harness) 将提示、记忆、可执行技能和子智能体规范存储为类型化、版本化的状态 [19]。主智能体 (Prime Agent) 将这些机制与持久核函数、递归会话、恢复和完整轨迹捕获集成。

编码智能体与长时程评估。编码智能体运行时使用可执行动作、仓库工具、沙箱、事件历史和结构化角色分配，在多个交互步骤中解决任务 [11, 20, 27, 37 – 39, 41]。可执行基准和轨迹语料库测量长上下文或扩展交互下的问题解决、指令遵循、检索和推理 [3, 5, 6, 13, 23, 25, 33, 40]。主智能体 (Prime Agent) 使执行基底持久且递归，然后记录根会话和后代会话的支出。

ARC-AGI-3 上的交互式推理。ARC-AGI-3 将抽象推理扩展到具有隐藏动态、目标和动作语义的交互式环境中 [1, 2]。社区系统构建并验证可执行的世界模型，将智能体表示为有状态的 Python 对象，优化外部工作空间，协调专用智能体，并在游戏间保留程序 [7, 9, 10, 19, 28, 29]。Prime Agent 为同类长时程交互任务提供持久的递归执行和标准化的评估设置。

多智能体与人机通信。语言模型智能体系统通过角色提示、自然语言消息、共享工件和显式信念状态进行协调 [11, 20, 21, 27, 29, 39]。学习型多智能体通信研究稀疏消息选择、压缩表示、社会学习、策略对齐和可解释性。

对于人类伙伴 [14 – 17]。主智能体通过持久的家庭作用域队列实现直接的智能体间通信，并将同一会话树暴露给人类进行检查和干预。

5 结论

Prime Agent 为智能体框架设计引入了一种新范式，其中持久执行、递归会话、自主控制、记录历史与持续框架共同构成长期任务的基础。在交互式推理、长上下文任务、自主研究、系统构建及持久环境中的结果表明，该基础在标准化执行与核算下支持不同形式的测试时计算。尽管相对于其他框架取得了成果，模型在决定如何分配子智能体、管理保留信息及优化可复用状态时仍面临阻力。许多框架能力仍未得到充分利用，因为当前模型并未经过操作这些能力的训练。我们预期模型与框架的协同学习将成为通往新长期能力的主要途径。直接使用 Prime Agent 进行训练可以教会模型更有效地利用集成框架，而对 RLM 和持续框架组件进行针对性训练则可以分离出它们各自的贡献。

致谢

我们感谢 Florian Brand、SinatraS 和 Patience Cave 在更多环境中帮助完成 Prime Agent 运行。SinatraS 创建了 PMPP Hard，Patience Cave 创建了 MazeBench。

参考文献

- [1] ARC Prize Foundation. ARC-AGI-3: A new challenge for frontier agentic intelligence, 2026. URL <https://arxiv.org/abs/2603.24621>.
- [2] ARC Prize Foundation. ARC-AGI community leaderboard. <https://arcprize.org/leaderboard/community>, 2026. Accessed August 2026.
- [3] Yushi Bai, Shangqing Tu, Jiajie Zhang, Hao Peng, Xiaozhi Wang, Xin Lv, Shulin Cao, Jiazheng Xu, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. LongBench v2: Towards deeper understanding and reasoning on realistic long-context multitasks, 2024. URL <https://arxiv.org/abs/2412.15204>.
- [4] Elie Bakouch and Prime Intellect. Measuring autonomous AI research. *Prime Intellect Blog*, August 2026. URL <https://www.primeintellect.ai/blog/measuring-autonomous-research>.
- [5] Amanda Bertsch, Adithya Pratapa, Teruko Mitamura, Graham Neubig, and Matthew R. Gormley. Oolong: Evaluating long context reasoning and aggregation capabilities, 2025. URL <https://arxiv.org/abs/2511.02817>.
- [6] Ziyang Chen, Xing Wu, Junlong Jia, Chaochen Gao, Qi Fu, Debing Zhang, and Songlin Hu. LongBench Pro: A more realistic and comprehensive bilingual long-context evaluation benchmark, 2026. URL <https://arxiv.org/abs/2601.02872>.
- [7] David Courtis, Wenhao Li, and Scott Sanner. OPINE-World: Programmatic world modeling with ontology-error-prioritized interactive exploration for ARC-AGI-3, 2026. URL <https://arxiv.org/abs/2607.01531>.
- [8] Tom Cunningham. Metrics of agent ability. <https://metr.org/notes/2026-07-24-metrics-of-model-ability/>, 07 2026.
- [9] Alexis Fox, Junlin Wang, Paul Rosu, and Bhuwan Dhingra. PRO-LONG: Programmatic memory enables long-horizon reasoning, 2026. URL <https://arxiv.org/abs/2607.20064>.

- [10] Paul Furgale, Severin Klingler, James Nolan, Matt Staats, Gaia Di Lorenzo, Elisa Martinez Abad, Christian Schuller, Razvan Dinu, Alessio Devoto, Pascal Berard, Gal Kaplun, Elad Sarafian, Riccardo Roveri, Leon Derczynski, and Ricardo Silveira Cabral. NVIDIA-labs OO Agents: Native python object-oriented agents, 2026. URL <https://arxiv.org/abs/2607.20709>.
- [11] Sirui Hong, Mingchen Zhuge, Jiaqi Chen, Xiwu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for A multi-agent collaborative framework, 2023. URL <https://arxiv.org/abs/2308.00352>.
- [12] Jack Hopkins, Mart Bakler, and Akbir Khan. Factorio learning environment, 2025. URL <https://arxiv.org/abs/2503.09617>.
- [13] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. SWE-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=VTF8yNQm66>.
- [14] Seth Karten. *Emergent Communication and Decision-Making in Multi-Agent Teams*. PhD thesis, Carnegie Mellon University, Pittsburgh, PA, 2023.
- [15] Seth Karten, Siva Kailas, Huao Li, and Katia Sycara. On the role of emergent communication for social learning in multi-agent reinforcement learning. In *Proceedings of the International Conference on Autonomous Agents and Multiagent Systems*, pages 2391–2393, 2023. doi: 10.5555/3545946.3598944. URL <https://arxiv.org/abs/2302.14276>.
- [16] Seth Karten, Mycal Tucker, Siva Kailas, and Katia Sycara. Towards true lossless sparse communication in multi-agent systems. In *IEEE International Conference on Robotics and Automation*, pages 7191–7197, 2023. doi: 10.1109/ICRA48891.2023.10161322. URL <https://arxiv.org/abs/2212.00115>.
- [17] Seth Karten, Mycal Tucker, Huao Li, Siva Kailas, Michael Lewis, and Katia Sycara. Interpretable learned emergent communication for human-agent teams. *IEEE Transactions on Cognitive and Developmental Systems*, 15(4):1801–1811, December 2023. doi: 10.1109/TCDS.2023.3236599. URL <https://doi.org/10.1109/TCDS.2023.3236599>.
- [18] Seth Karten, Alex L. Zhang, and Sami Jaghouar. Emulator Bench: Verifiable Whole-System Emulation for Ultra Long-Horizon Coding Agents, 2026. Manuscript.
- [19] Seth Karten, Joel Zhang, Tersoo Upaa, Ruihong Feng, Wenzhe Li, Chengshuai Shi, Chi Jin, and Kiran Vodrahalli. Continual harness: Online adaptation for self-improving foundation agents, 2026. URL <https://arxiv.org/abs/2605.09998>.
- [20] Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. CAMEL: Communicative agents for “mind” exploration of large scale language model society, 2023. URL <https://arxiv.org/abs/2303.17760>.
- [21] Huao Li, Yu Chong, Simon Stepputtis, Joseph P. Campbell, Dana Hughes, Charles Lewis, and Katia Sycara. Theory of mind for multi-agent collaboration via large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 180–192, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.13. URL <https://aclanthology.org/2023.emnlp-main.13/>.
- [22] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Sean Welleck, Bodhisattwa Prasad Majumder, Shashank Gupta, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, 2023. URL <https://arxiv.org/abs/2303.17651>.
- [23] Sumeet Ramesh Motwani, Daniel Nichols, Charles London, Peggy Li, Fabio Pizzati, et al. LongCoT: Benchmarking long-horizon chain-of-thought reasoning, 2026. URL <https://arxiv.org/abs/2604.14140>.
- [24] Charles Packer, Vivian Fang, Shishir G. Patil, Kevin Lin, Sarah Wooders, and Joseph E.

- Gonzalez. MemGPT: Towards llms as operating systems, 2023. URL <https://arxiv.org/abs/2310.08560>.
- [25] Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yizhe Zhang. Training software engineering agents and verifiers with SWE-Gym, 2024. URL <https://arxiv.org/abs/2412.21139>.
- [26] Joon Sung Park, Joseph O’Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *ACM Symposium on User Interface Software and Technology*, 2023. URL <https://arxiv.org/abs/2304.03442>.
- [27] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. ChatDev: Communicative agents for software development, 2023. URL <https://arxiv.org/abs/2307.07924>.
- [28] Sergey Rodionov. Executable world models for ARC-AGI-3 in the era of coding agents, 2026. URL <https://arxiv.org/abs/2605.05138>.
- [29] Elad Sarafian, Gal Kaplun, Ron Banner, Daniel Soudry, and Boris Ginsburg. Workspace optimization: How to train your agent, 2026. URL <https://arxiv.org/abs/2605.09650>.
- [30] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, 2023. URL <https://arxiv.org/abs/2302.04761>.
- [31] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, 2023. URL <https://arxiv.org/abs/2303.11366>.
- [32] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters, 2024. URL <https://arxiv.org/abs/2408.03314>.
- [33] Diane Tchuindjo, Devavrat Shah, and Omar Khattab. OBLIQ-Bench: Exposing overlooked bottlenecks in modern retrievers with latent and implicit queries, 2026. URL <https://arxiv.org/abs/2605.06235>.
- [34] Alan M. Turing. On computable numbers, with an application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society*, 42(1):230–265, 1936. doi: 10.1112/plms/s2-42.1.230.
- [35] John von Neumann. First draft of a report on the EDVAC. Technical report, Moore School of Electrical Engineering, University of Pennsylvania, June 1945.
- [36] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models, 2023. URL <https://arxiv.org/abs/2305.16291>.
- [37] Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. CodeAct: Executable code actions elicit better llm agents, 2024. URL <https://arxiv.org/abs/2402.01030>.
- [38] Xingyao Wang et al. OpenHands: An open platform for ai software developers as generalist agents, 2024. URL <https://arxiv.org/abs/2407.16741>.
- [39] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling next-gen llm applications via multi-agent conversation, 2023. URL <https://arxiv.org/abs/2308.08155>.
- [40] Yiheng Xu, Dunjie Lu, Zhennan Shen, Junli Wang, Zekun Wang, Yuchen Mao, Caiming Xiong, and Tao Yu. AgentTrek: Agent trajectory synthesis via guiding replay with web tutorials, 2024. URL <https://arxiv.org/abs/2412.09605>.

- [41] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, 2024. doi: 10.48550/arXiv.2405.15793. URL <https://arxiv.org/abs/2405.15793>.
- [42] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. URL <https://arxiv.org/abs/2210.03629>.
- [43] Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. STaR: Bootstrapping reasoning with reasoning. In *Advances in Neural Information Processing Systems*, volume 35, 2022. URL <https://arxiv.org/abs/2203.14465>.
- [44] Alex L. Zhang, Tim Kraska, and Omar Khattab. Recursive language models, 2025. URL <https://arxiv.org/abs/2512.24601>.
- [45] Jingyu Zhang, Tianjian Li, William Jurayj, Hongyuan Zhan, Benjamin Van Durme, and Daniel Khashabi. Many-tier instruction hierarchy in llm agents, 2026. URL <https://arxiv.org/abs/2604.09443>.

A 纳米 GPT 极速跑中的示例外循环实验

以下摘录均为智能体在纳米 GPT 运行期间（第 3.3 节）于基准训练脚本之外创建并运行的实验，从轨迹中复现并裁剪长度。Kimi K3 利用全局优化器重新推导了牛顿-舒尔茨迭代系数，并逐位精确检查了 bf16 舍入：

```
from scipy.optimize import differential_evolution
grid_in = np.concatenate([np.linspace(0.02, 0.05, 10),
                           np.linspace(0.05, 1.0, 190)])
grid_over = np.linspace(1.0, 1.3, 20)

def p_map(sig, a, b, c, iters=6):
    x = sig
    for _ in range(iters):
        x = a*x + b*x**3 + c*x**5
    return x

def objective(params):
    a, b, c = params
    dev = np.max(np.abs(p_map(grid_in, a, b, c) - 1.0))
    over = max(0.0, np.max(np.abs(p_map(grid_over, a, b, c))) - 1.15)
    return dev + 5.0*over

res = differential_evolution(objective,
    [(1.0, 6.0), (-8.0, 0.0), (0.0, 5.0)],
    maxiter=300, tol=1e-9, seed=0, polish=True)
```

DeepSeek V4 Pro 构建了一个训练问题的校准玩具模型，其中小批量噪声由真实克罗内克黑塞矩阵塑造，并配备自然梯度预言机臂：

```
"""Calibrated toy: Kron-quadratic + CORRECT Kron-Hessian
minibatch noise. eps = Hl^{1/2} Z Hr^{1/2} / sqrt(n_eff).
Ideal preconditioner: Ql = Hl^{-1/2}, Qr = Hr^{-1/2}."""
G = Hl @ W @ Hr
eps = Hl12 @ torch.randn(p, q) @ Hr12 / (n_eff ** 0.5)
```

```
g = G + eps
elif opt == "natgrad": # oracle arm
    E1, V1 = torch.linalg.eigh(H1)
    Er, Vr = torch.linalg.eigh(Hr)
    u = V1 @ (V1.T @ d @ Vr /
              (E1[:, None]**0.5 * Er[None, :]**0.5)) @ Vr.T
```

GLM 5.3 在任何 GPU 屏幕之前于 CPU 上调试了其 SOAP 实现:

```
torch.manual_seed(0)
for shape in [(768, 768), (3072, 768), (768, 3072)]:
    p = torch.nn.Parameter((torch.randn(*shape) * 0.02).bfloat16())
    opt = SOAP([p], lr=0.025)
    for t in range(25):
        p.grad = (torch.randn(*shape) * 0.01).to(torch.bfloat16)
        opt.step()
        if not torch.isfinite(p.data).all():
            print(shape, 'NaN at step', t+1)
            st = opt.state[p]
            print('L finite', torch.isfinite(st['L']).all().item(),
                  'v finite', torch.isfinite(st['v']).all().item())
            break
```

B 示例程序化编排

```
# Admit independent subagents; do not wait for answers here.
review = await rlm("Audit the implementation. Reply with concrete issues.",
                   name="reviewer")
tests = await rlm("Run the test suite and classify failures.",
                  name="tester")

# Later, recover retained sessions and send a follow-up.
children = await rlm.list_subagents()
await agent_message.send(
    "Also inspect error-handling edge cases.",
    receiver_role="child", receiver_name=review.name)
```

显式回复路径是有意为之：子会话是一个持久的并发会话，而非由 rlm 返回的无状态完成。

C 大语言模型使用披露

在本文稿的准备过程中，使用了大语言模型来辅助代码开发、文字润色和格式调整。所有科学论断、实验设计、分析以及学术贡献均完全由作者完成。